

Sistemas Operativos 2026



Clase 1

U1 Introducción – Llamadas al Sistema

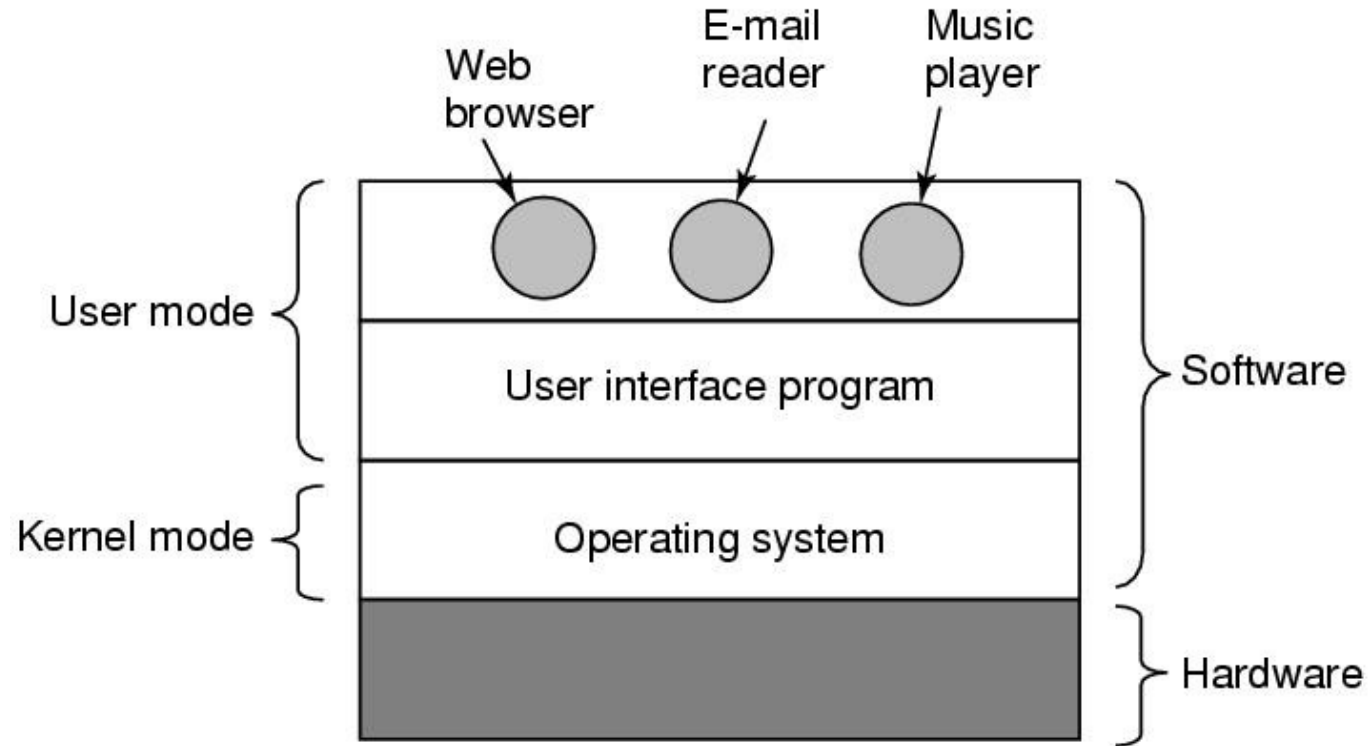
Ing. Juan Manuel Calvo

¿Qué es el sistema operativo?

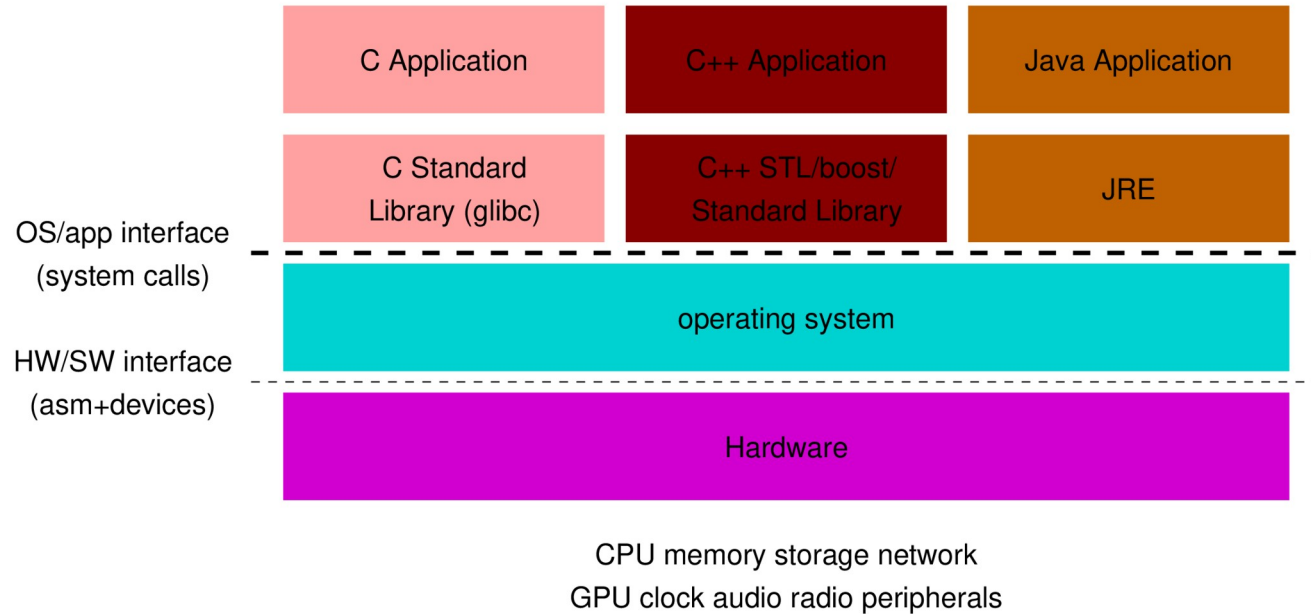
- Un conjunto de programas que actúan como intermediarios entre las aplicaciones y el hardware de la computadora.



Estructura de un sistema informático



Estructura de un sistema informático II



Sistemas operativos - Función

- La función de un sistema operativo es compartir un ordenador entre múltiples programas y proporcionar un conjunto de servicios más útil que el que ofrece el hardware por sí solo.
- Un sistema operativo gestiona y abstrae el hardware de bajo nivel, de modo que, por ejemplo, un procesador de textos no necesita preocuparse por el tipo de disco duro que se utiliza.
- Un sistema operativo comparte el hardware entre múltiples programas para que se ejecuten (o parezcan ejecutarse) simultáneamente.
- Finalmente, los sistemas operativos proporcionan mecanismos controlados para que los programas interactúen, permitiéndoles compartir datos o trabajar juntos.

Sistemas Operativos - Uso

- Un sistema operativo ofrece servicios a los programas de usuario a través de una interfaz.
- Diseñar una buena interfaz resulta complejo.
- Por un lado, deseamos que la interfaz sea sencilla y limitada, ya que esto facilita una implementación correcta.
- Por otro lado, podemos sentir la tentación de ofrecer muchas funciones sofisticadas a las aplicaciones.
- La clave para resolver esta disyuntiva reside en diseñar interfaces que se basen en unos pocos mecanismos que puedan combinarse para ofrecer una gran versatilidad.

¿Para qué usamos el sistema operativo?

- **Máquina extendida**
 - Interfaz de llamadas al sistema - abstracta y simplificada
 - Portabilidad - los programas no necesitan tener en cuenta los detalles del medio ambiente.
 - Manejo de dispositivos
 - Comunicaciones entre procesos y a otras máquinas
- **Administrador de recursos**
 - Procesos - esconden a los programas unos de otros
 - Policía de tráfico - manejo de recursos, administra quien corre y cuando.
 - Manejo de memoria - protege de las equivocaciones de otros programas.
 - Seguridad - protege de la malicia o los errores de otros programas

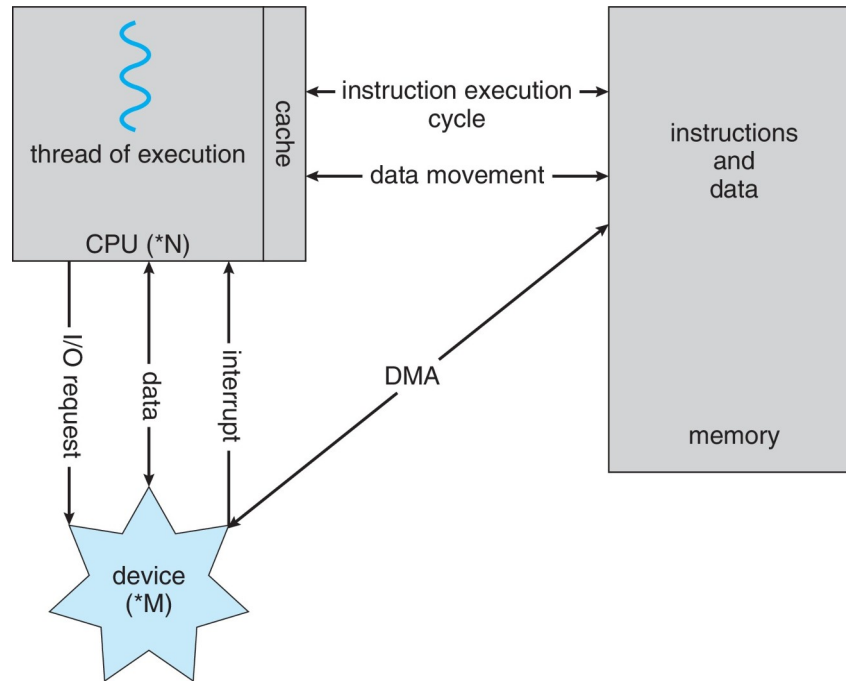
Definición del sistema operativo

- No hay una definición aceptada universalmente.
 - Varía enormemente
 - Una buena aproximación es “Todo lo que envía el proveedor cuando Ud. compra un sistema operativo”.
- El **kernel** (una parte del sistema operativo) es “el único programa que se está ejecutando en todo momento en la computadora”.
- Otros son:
 - **Programas del sistema** (los que se incluyen en el sistema operativo, pero no forman parte del kernel),
 - **Programas de aplicación**, todos los demás
- Los sistemas operativos actuales, tanto los de uso general y los de computación móvil también incluyen **middleware**:
 - conjunto de software que brinda servicios adicionales a los desarrolladores de aplicaciones, como bases de datos, multimedia, gráficos.

Sistemas operativos - funcionamiento

- El **kernel** es un programa especial que proporciona servicios a los programas en ejecución.
- A cada programa en ejecución, llamado **proceso**, se le asigna memoria que contiene:
 - Las **instrucciones** implementan los cálculos del programa.
 - Los **datos** son las variables sobre las que actúa el cálculo.
 - Un **stack** organiza las llamadas a procedimientos del programa.
- Una computadora suele tener muchos procesos, pero un único kernel.
- Cuando un proceso necesita invocar un servicio del kernel, realiza una **llamada al sistema**.
 - La llamada al sistema entra en el kernel; este realiza el servicio y regresa.
- De esta forma, un proceso alterna entre ejecutarse en el espacio de usuario y en el espacio del kernel.
- El kernel utiliza los mecanismos de **protección de hardware** proporcionados por la CPU para garantizar que cada proceso que se ejecuta en el espacio de usuario solo pueda acceder a la memoria que tiene asignada.
- El kernel se ejecuta con los privilegios de hardware necesarios para implementar estas protecciones; los programas de usuario se ejecutan sin dichos privilegios.
- Cuando un programa de usuario invoca una llamada al sistema, el hardware eleva el nivel de privilegios y comienza a ejecutar una función predefinida en el kernel.
- El conjunto de llamadas al sistema que proporciona un núcleo constituye la interfaz que ven los programas de usuario.

Como funciona una computadora moderna



Operaciones del sistema operativo

- **Bootstrap:** código simple para inicializar el sistema y cargar el kernel
- Se carga el kernel
- Se Inician los demonios del sistema (servicios provistos fuera del kernel)
- El kernel es guiado por interrupciones (hardware y software)
 - Interrupciones de hardware por un dispositivo
 - Interrupciones de software (excepción o trap):
 - Error de software (p. ej., división por cero)
 - Solicitud de servicio del sistema operativo: llamada al sistema
 - Otros problemas de los procesos incluyendo los bucles infinitos, procesos que modifican a otros o al sistema operativo.

Multiprogramación

- Una sola aplicación no siempre puede mantener la CPU y los dispositivos de E/S ocupados.
- La multiprogramación organiza trabajos (código y datos) para que la CPU siempre tenga algo para ejecutar.
- Un subconjunto del total de trabajos en el sistema se mantiene en la memoria
- El trabajo es seleccionado y ejecutado por la planificación (job scheduling).
- Cuando un trabajo tiene que esperar (para E/S, por ejemplo), el sistema operativo cambia a otro trabajo

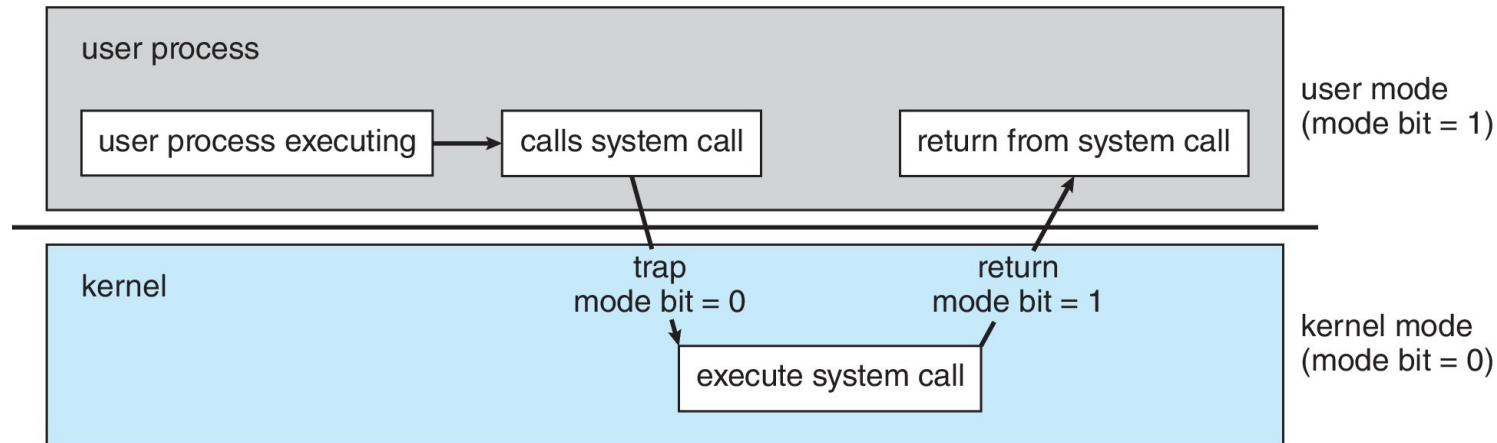
Multitarea (tiempo compartido)

- La CPU cambia de tarea con tanta frecuencia que los usuarios pueden interactuar con cada tarea mientras se ejecuta, creando una computación **interactiva**.
 - El **tiempo de respuesta** debe ser < 1 segundo
 - Cada usuario tiene al menos un programa ejecutándose en memoria => proceso
 - Al mismo tiempo varios trabajos están listos para ejecutarse => planificación de la CPU
 - Si los procesos no caben en la memoria, el intercambio los mueve dentro y fuera para que se ejecuten
 - La **memoria virtual** permite la ejecución de procesos que no están completamente en la memoria

Operación de modo dual

- La operación de **modo dual** permite que el sistema operativo se proteja a sí mismo y a otros componentes del sistema
 - Modo **usuario** y modo **kernel**
- El hardware proporciona un **Bit de modo**
 - Brinda la capacidad de distinguir cuándo el sistema está ejecutando código de usuario o código kernel.
 - Cuando un usuario está ejecutando => el bit de modo es "usuario"
 - Cuando el código del kernel se está ejecutando => el bit de modo es "kernel"
- ¿Cómo garantizamos que el usuario no establezca explícitamente el bit de modo en "kernel"?
 - Algunas instrucciones se designan como **privilegiadas**, y son solo ejecutables en modo kernel.
 - Sólo se puede cambiar al modo kernel con una **llamada al sistema**, el retorno de esta lo restablece a modo usuario.

Transición del Modo Usuario al Modo Kernel



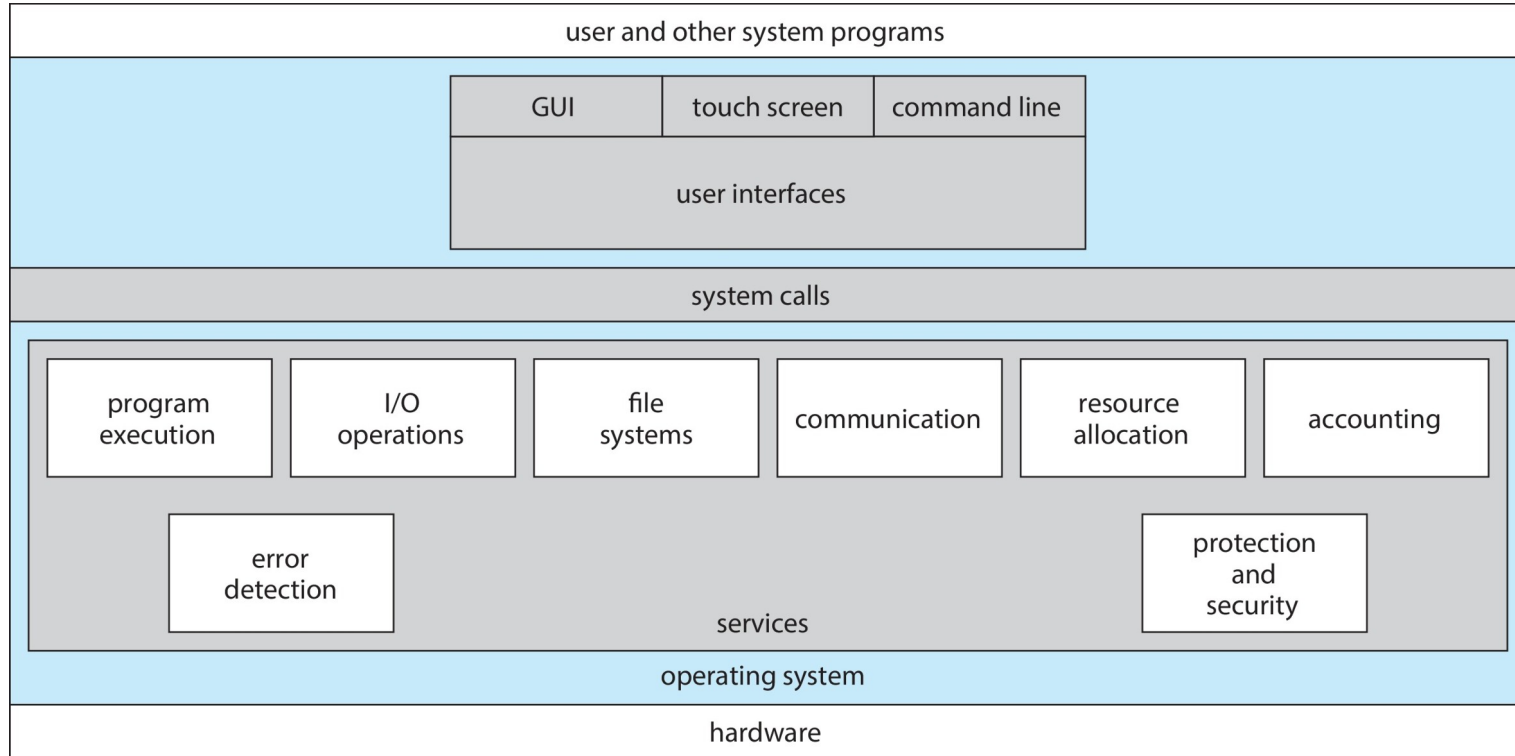
Timer

- Para evitar bucles infinitos (o procesos que acaparen recursos)
- El timer está configurado para interrumpir la computadora después de un período de tiempo
- Tiene un contador que es decrementado por el reloj físico.
- El sistema operativo establece el contador (esto requiere una instrucción privilegiada).
- Cuando el contador llega a 0 genera una interrupción.
- El kernel lo configura antes de transferir el control al proceso para poder recuperarlo o terminar el programa que exceda el tiempo asignado

Gestión de procesos

- Un proceso es un **programa en ejecución**. Es una unidad de trabajo dentro del sistema.
- Un programa es una entidad pasiva; proceso es una entidad activa.
- El proceso necesita recursos para realizar su tarea.
 - CPU, memoria, E/S, archivos
 - Datos de inicialización
- La terminación del proceso requiere la recuperación de cualquier recurso reutilizable
- Los procesos de un solo hilo (**Single-threaded**) tienen un **contador de programa** que especifica la ubicación de la próxima instrucción a ejecutar
 - El proceso ejecuta las instrucciones secuencialmente, una a la vez, hasta completarse
- El proceso multihilo (**Multi-threaded**) tiene un **contador de programa por hilo**.
- Por lo general, el sistema tiene muchos procesos, algunos de usuarios, otros del sistema operativo que se ejecutan simultáneamente en una o más CPU
 - Concurrencia multiplexando las CPUs entre los procesos/hilos

Una vista de los servicios del sistema operativo



Servicios del sistema operativo

- Los sistemas operativos proporcionan un entorno para la ejecución de programas y servicios a programas y usuarios.
- Un conjunto de servicios del sistema operativo proporciona funciones que son útiles para el usuario:
- Interfaz de usuario: línea de comandos (CLI), interfaz gráfica de usuario (GUI), pantalla táctil, sistema por lotes (Batch)
- Ejecución del programas: cargarlo en la memoria y ejecutarlo.
- Operaciones de E/S: un programa en ejecución puede requerir E/S, a archivos o a dispositivos.
- Manipulación del sistema de archivos: Los programas necesitan leer y escribir archivos y directorios, crearlos y eliminarlos, buscarlos, listar información de archivos, administración de permisos.
- Comunicaciones: los procesos pueden intercambiar información a través de una red. Las comunicaciones pueden ser por memoria compartida o paso de mensajes
- Detección de errores: errores de hardware de la CPU, memoria, dispositivos de E/S, en el programa del usuario

Intérprete de línea de comandos

- CLI permite la entrada directa de comandos.
- A veces implementado en kernel, a veces un programa del sistema
 - Frecuentemente hay más de un shell
- Su función es obtener comandos del usuario y ejecutarlos.
- A veces los comandos están incorporados, a veces son sólo nombres de programas.
 - Si es esto último, agregar nuevos comandos no requiere modificar el shell.

Linux CLI - Bash

```
jmc@raspberrypi: ~  
jmc:~$ raspi  
Warning: No xauth data; using fake authentication data for X11 forwarding.  
Linux raspberrypi 5.15.32+ #1538 Thu Mar 31 19:37:58 BST 2022 armv6l  
Last login: Wed Jul 27 19:02:22 2022 from 192.168.0.172  
jmc@raspberrypi:~ $ uptime  
19:03:56 up 27 min, 1 user, load average: 0.24, 0.20, 0.25  
jmc@raspberrypi:~ $ df -h  
Filesystem      Size  Used Avail Use% Mounted on  
/dev/root        7.1G  2.2G  4.6G  32% /  
devtmpfs         111M   0  111M   0% /dev  
tmpfs            239M   0  239M   0% /dev/shm  
tmpfs            96M   580K  95M   1% /run  
tmpfs            5.0M   4.0K  5.0M   1% /run/lock  
/dev/mmcblk0p1   253M   50M  203M  20% /boot  
tmpfs            48M    0   48M   0% /run/user/1000  
jmc@raspberrypi:~ $ ls -ltr bmp280-python/  
total 28  
-rw-r--r-- 1 jmc jmc  893 Jul  8 18:24 README.md  
-rw-r--r-- 1 jmc jmc 2483 Jul  8 18:24 Makefile  
-rw-r--r-- 1 jmc jmc 1070 Jul  8 18:24 LICENSE  
-rwxr-xr-x 1 jmc jmc  365 Jul  8 18:24 install.sh  
-rwxr-xr-x 1 jmc jmc  385 Jul  8 18:24 uninstall.sh  
drwxr-xr-x 7 jmc jmc 4096 Jul  8 18:26 library  
drwxr-xr-x 2 jmc jmc 4096 Jul  8 20:19 examples  
jmc@raspberrypi:~ $
```

System Calls

- Interfaz de programación para los servicios proporcionados por el sistema operativo
- Normalmente escrito en un lenguaje de alto nivel (C o C++)
- Mayormente accedido por programas a través de una interfaz de programación de aplicaciones (**API**) de alto nivel en lugar del uso directo de llamadas al sistema.
- Las APIs se implementan como bibliotecas de funciones (libraries)
- Las tres API más comunes son:
 - **Win32** o **WinAPI** para Windows
 - **POSIX** para sistemas basados en UNIX, Linux y mac OS
 - **Java Standard Library** para la máquina virtual Java (JVM)

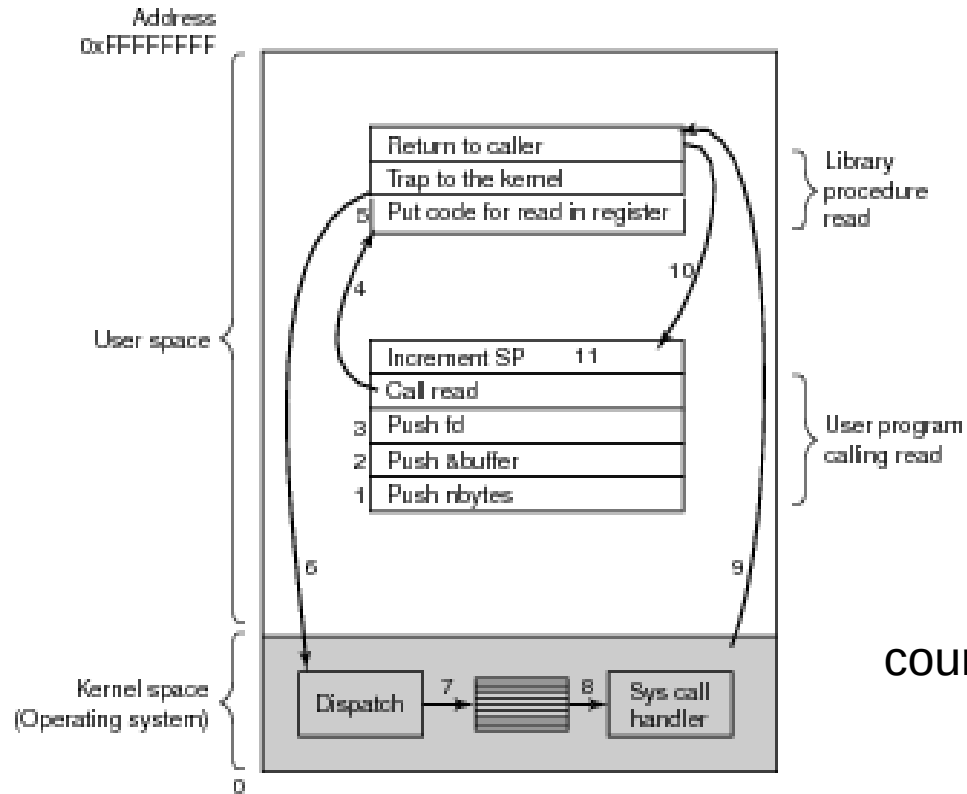
XV6 Syscalls

System call	Description
int fork()	Create a process, return child's PID.
int exit(int status)	Terminate the current process; status reported to wait(). No return.
int wait(int *status)	Wait for a child to exit; exit status in *status; returns child PID.
int kill(int pid)	Terminate process PID. Returns 0, or -1 for error.
int getpid()	Return the current process's PID.
int pause(int n)	Pause for n clock ticks.
int exec(char *file, char *argv[])	Load a file and execute it with arguments; only returns if error.
char *sbrk(int n)	Grow process's memory by n zero bytes. Returns start of new memory.
int open(char *file, int flags)	Open a file; flags indicate read/write; returns an fd (file descriptor).
int write(int fd, char *buf, int n)	Write n bytes from buf to file descriptor fd; returns n.
int read(int fd, char *buf, int n)	Read n bytes into buf; returns number read; or 0 if end of file.
int close(int fd)	Release open file fd.
int dup(int fd)	Return a new file descriptor referring to the same file as fd.
int pipe(int p[])	Create a pipe, put read/write file descriptors in p[0] and p[1].
int chdir(char *dir)	Change the current directory.
int mkdir(char *dir)	Create a new directory.
int mknod(char *file, int, int)	Create a device file.
int fstat(int fd, struct stat *st)	Place info about an open file into *st.
int link(char *file1, char *file2)	Create another name (file2) for the file file1.
int unlink(char *file)	Remove a file.



UNIVERSIDAD
DEL CEMA
UCEMA

Ejemplo de system call



```
count = read(fd, buffer, nbytes);
```

Ejemplo de llamadas al sistema

Considere la función `read( )` que está disponible en los sistemas UNIX y Linux. La API de esta función se obtiene con:

```
$ man 2 read
```

Esta es la descripción de esta API

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count);
```

Un programa que use la función `read()` debe incluir el archivo de encabezado `unistd.h`, ya que este archivo define (entre otras cosas) los tipos de datos `ssize_t` y `size_t`. Los parámetros pasados a `read()` son los siguientes:

- `int fd`: el descriptor del archivo a leer
- `void *buf`: un búfer en el que se leerán los datos
- `size_t count`: el número máximo de bytes que contendrá el búfer

En una lectura exitosa, se devuelve el número de bytes leídos. Un valor de retorno de 0 indica el final del archivo. Si ocurre un error, `read()` devuelve `-1`



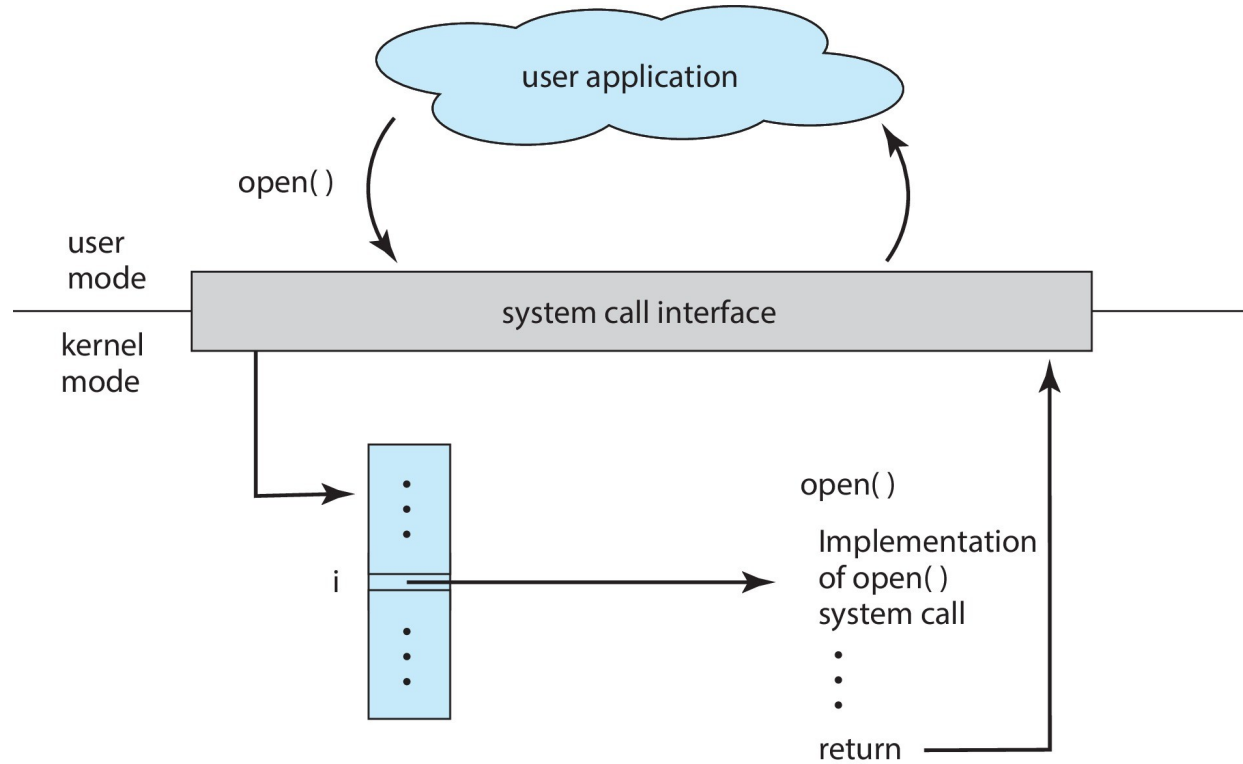
UNIVERSIDAD
DEL CEMA

UCEMA

Implementación de las llamadas al sistema

- A cada llamada al sistema se asocia un número
 - Las llamadas al sistema se hacen usando ese número.
- La interfaz de llamada al sistema invoca la llamada al sistema prevista en el kernel del sistema operativo y devuelve el estado de la llamada al sistema y los valores de retorno.
- El proceso que llama no necesita saber cómo se implementa la llamada al sistema.
 - Solo necesita obedecer la API y comprender qué hará el sistema operativo como resultado de la llamada
 - La mayoría de los detalles de la interfaz del sistema operativo están ocultos para el programador por API
 - Administrado por la biblioteca de soporte en tiempo de ejecución (conjunto de funciones integradas en las bibliotecas incluidas con el compilador)

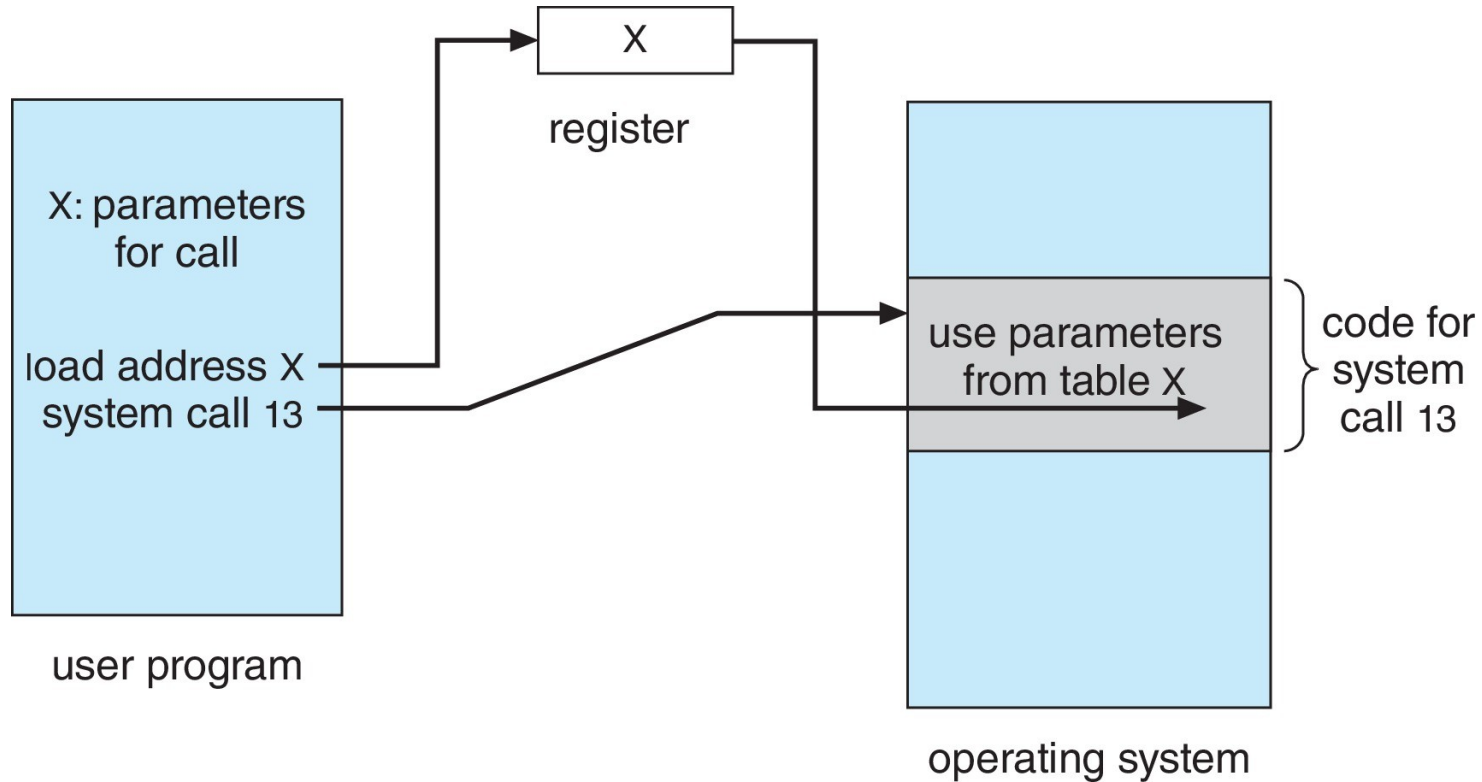
Relaciones entre la API – System call – Sistema operativo



Llamadas al sistema - Paso de parámetros

- Normalmente se requiere información adicional al número de llamada al sistema.
 - El tipo exacto y la cantidad de información varían según el sistema operativo y la llamada
- Tres métodos generales utilizados para pasar parámetros al sistema operativo
 - Más simple: pasar los parámetros en registros
 - En algunos casos, puede haber más parámetros que registros.
 - Parámetros almacenados en un bloque, o tabla, en la memoria y la dirección del bloque pasada como parámetro en un registro
 - Este enfoque adoptado por Linux y Solaris
 - Parámetros colocados o empujados en la pila por el programa y sacados de la pila por el sistema operativo
 - Los métodos de bloque y pila no limitan el número o la longitud de los parámetros que se pasan

Paso de parámetros via una tabla



Tipos de llamadas al sistema

- **Control de procesos**

- crear proceso, terminar proceso
- finalizar, abortar
- cargar, ejecutar
- obtener atributos de proceso, establecer atributos de proceso
- Esperar un tiempo, un evento, una señal
- asignar y liberar memoria
- Volcar memoria si hay error, depurador para determinar errores, ejecución de un solo paso
- Bloqueos para administrar el acceso a datos compartidos entre procesos

Tipos de llamadas al sistema (Cont)

- **Gestión de archivos**
 - crear archivo, eliminar archivo
 - abrir, cerrar archivo
 - leer, escribir, reposicionar
 - obtener y establecer atributos de archivo
- **Gestión de dispositivos**
 - Solicitud de dispositivo, liberación de dispositivo
 - leer, escribir, reposicionar
 - obtener atributos de dispositivo, establecer atributos de dispositivo
 - conectar o desconectar lógicamente los dispositivos

Tipos de llamadas al sistema (Cont)

- **Mantenimiento de la información**
 - obtener hora o fecha, establecer hora o fecha
 - obtener datos del sistema, establecer datos del sistema
 - obtener y establecer atributos de proceso, archivo o dispositivo
- **Comunicaciones**
 - crear, eliminar una conexión
 - enviar, recibir mensajes
 - En la memoria compartida crear y obtener acceso a regiones de memoria
 - información del estado de la transferencia
 - conectar y desconectar dispositivos remotos

Tipos de llamadas al sistema (Cont)

- **Protección**
 - Controlar el acceso a los recursos
 - Obtener y establecer permisos
 - Permitir y denegar el acceso de los usuarios



Ejemplos de llamadas al sistema

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

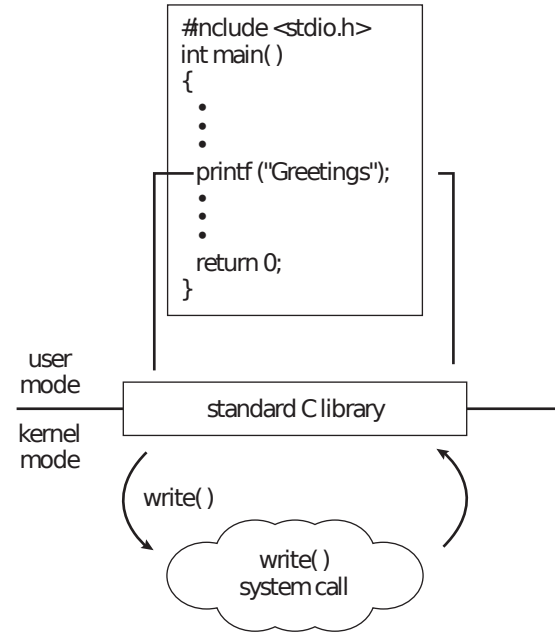
	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



UNIVERSIDAD
DEL CEMA
UCEMA

Ejemplo con biblioteca de C estandard

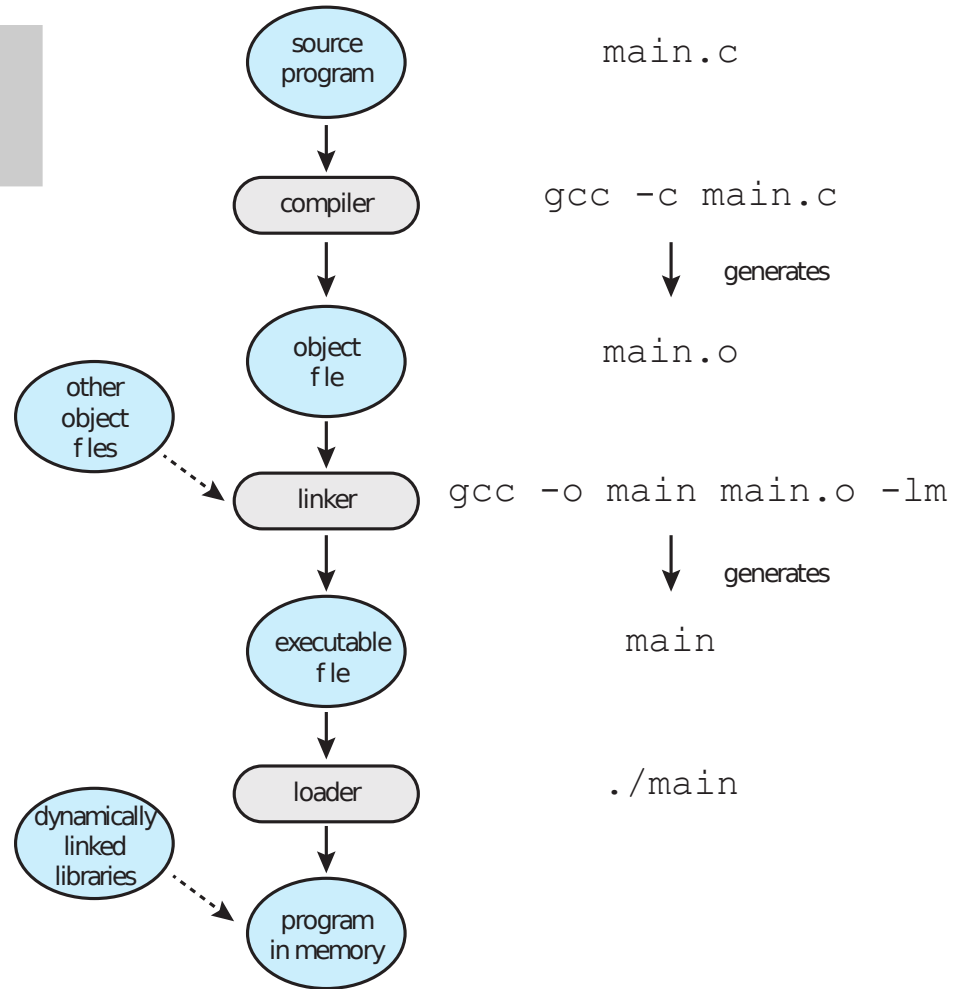
- Programa en C que invoca la llamada a la biblioteca printf(), la cual llama a la llamada al sistema write()



Enlazadores y cargadores (Linkers and Loaders)

- El código fuente es compilado en archivos de objeto diseñados para cargarse en cualquier ubicación de memoria física: **archivo objeto reubicable**
- El **linker** combina estos en un solo archivo ejecutable binario
 - También incluye las bibliotecas
- El programa reside en el almacenamiento secundario como ejecutable binario
- Debe ser traído a la memoria por el **loader** para ser ejecutado
 - La **reubicación** asigna direcciones finales a las partes del programa y ajusta el código y los datos en el programa para que coincidan con esas direcciones
- Los sistemas modernos de propósito general no vinculan bibliotecas en los ejecutables
 - Más bien, las bibliotecas vinculadas dinámicamente (en Windows, DLL) se cargan según sea necesario, compartidas por todos los que usan la misma versión de esa misma biblioteca (cargada una vez)
- Los archivos objeto y ejecutables tienen formatos estándar, por lo que el sistema operativo sabe cómo cargarlos e iniciarlos

El rol del enlazador y el cargador



Estructura del sistema UNIX y Linux

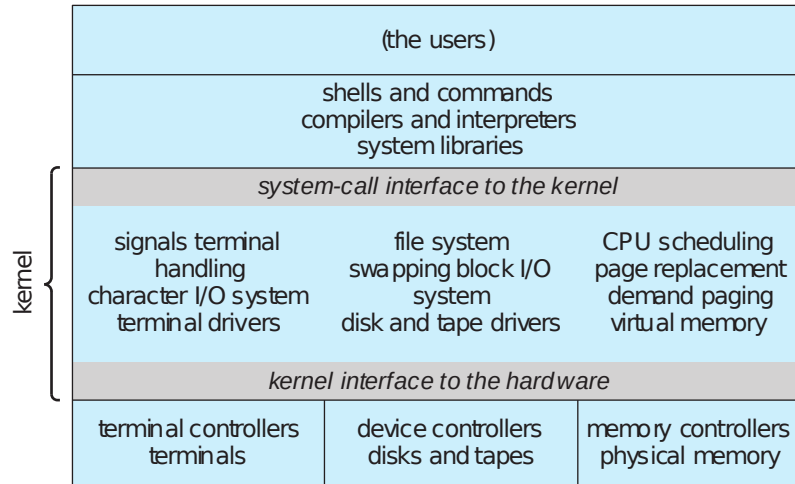


Figure 2.12 Traditional UNIX system structure.

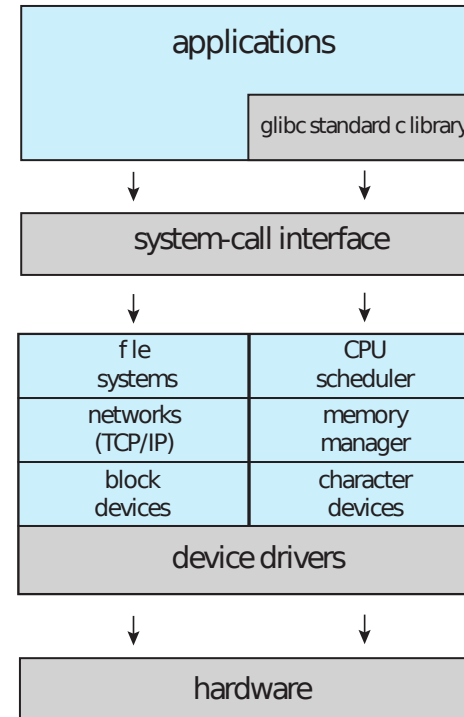
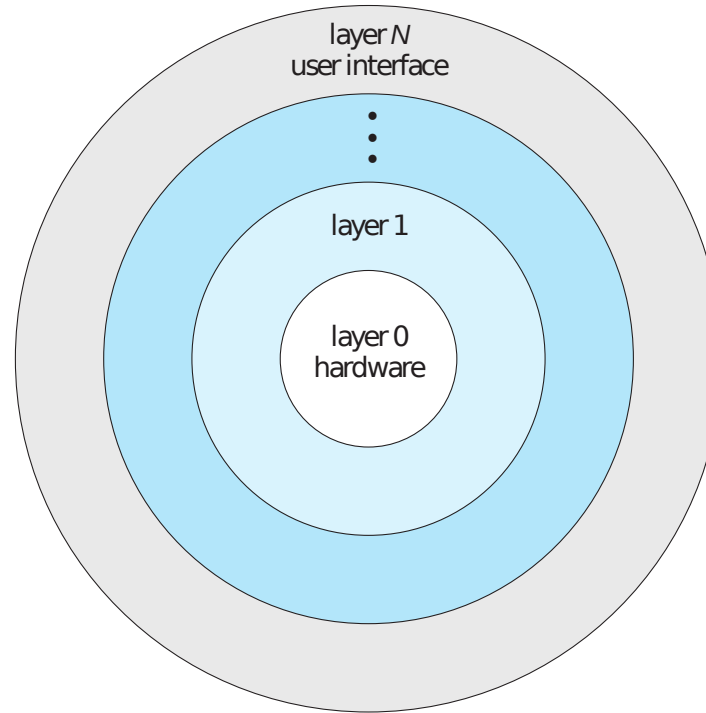


Figure 2.13 Linux system structure.

Enfoque en capas (layers)

- El sistema operativo está dividido en una serie de capas (niveles), cada una construida sobre las capas inferiores. La capa inferior (capa 0), es el hardware; la más alta (capa N) es la interfaz de usuario.
- Con la modularidad, las capas se seleccionan de modo que cada una use funciones (operaciones) y servicios solamente de la capa de nivel inferior



Risc-V Privilege levels

- At any time, a RISC-V hardware thread (hart) is running at some privilege level encoded as a mode in one or more CSRs (control and status registers).
- Three RISC-V privilege levels are currently defined

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	<i>Reserved</i>	
3	11	Machine	M

- Privilege levels are used to provide protection between different components of the software stack, and attempts to perform operations not permitted by the current privilege mode will cause an exception to be raised.
- These exceptions will normally cause traps into an underlying execution environment.



UNIVERSIDAD
DEL CEMA
UCEMA